

Unit 12: More on Templates

CONTENTS

Objectives

Introduction

12.1 Class Template and Inheritance

12.2 Recursion with Template Function

12.3 Preprocessors in C++

12.4 Macros in C++

Summary

Keywords

Self Assessment

Review Questions

Answers for Self Assessment

Further Readings

Objectives

After studying this unit, you will be able to:

- Describe the class template and inheritance
- Understand recursion with template function
- Understand preprocessors
- Define Macros

Introduction

A template can be considered as a kind of macro. When an object of a specific type is defined for actual use, the template definition for that class is substituted with the required data type. Since a template is defined with a parameter that would be replaced by a specified data type at the time of actual use of the class or function, the templates are sometimes called parameterized classes or functions.

12.1 Class Template and Inheritance

- It is possible to inherit from a template class. All the usual rules for inheritance and polymorphism apply.
- If we want the new, derived class to be generic it should also be a template class; and pass its template parameter along to the base class.



Example

```
#include<iostream>
using namespace std;
template <typename T>
```

```

class base {
protected:
    int x=100,y=10;
};

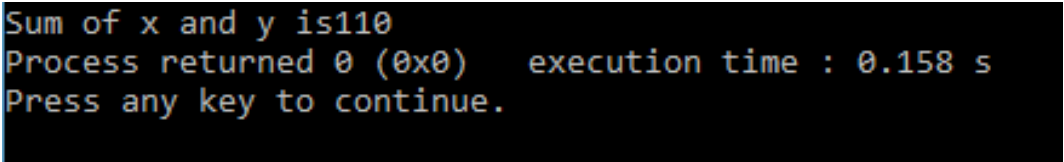
template <typename T>
class derived : public base<T> {

public:
    int sum() { return this->x+this->y; }
};

int main() {
    derived<int> d;
    cout<<"Sum of x and y is"<<d.sum();
    return 0;
}

```

Output



```

Sum of x and y is110
Process returned 0 (0x0)   execution time : 0.158 s
Press any key to continue.

```



Task: Write a program using C++ that demonstrate working of class template and inheritance.

12.2 Recursion with Template Function

- The basic purpose of templates in the C++ language is to support functions and classes parameterized by type. However, templates can also be used to encode arbitrarily complex computational procedures.
- In order to implement a procedure using templates, the procedure must be formulated recursively.



Example

```

#include <iostream>
using namespace std;
void tprintf(const char* format)
{
    cout << format;
}

template<typename T, typename... Targs>
void tprintf(const char* format, T value, Targs... Fargs)
{

```

```

for ( ; *format != '\0'; format++ ) {
    if ( *format == '%' ) {
        cout << value;
        tprintf(format+1, Fargs...);
        return;
    }
    cout << *format;
}
}
int main()
{
    tprintf("% Programming% %\n", "C++", "!", 123);
    return 0;
}

```

Output

```

C++ Programming! 123

Process returned 0 (0x0)   execution time : 0.017 s
Press any key to continue.

```

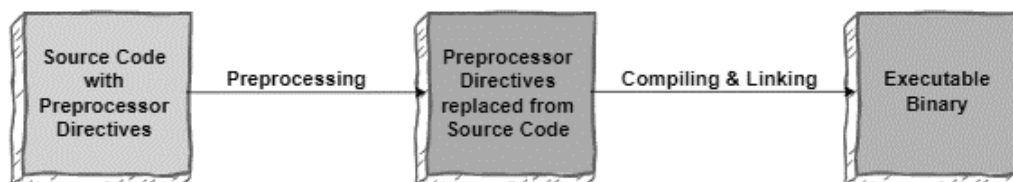


Task: Write a program using C++ that demonstrate working of recursion and template function.

12.3 Preprocessors in C++

The preprocessor is a directive given to the compiler before the compilation phase of the program. The compiler executes these directives before compiling the code into an executable file.

Take a look at the image below:



The image above illustrates that, after the Preprocessing phase, all preprocessor directives are replaced with C++ code in the source code file. Then, the compilation begins.



Did you know?

- Preprocessors are programs that process our source code before compilation.

- Preprocessor programs provide preprocessors directives which tell the compiler to preprocess the source code before compiling.
- All of these preprocessor directives begin with a '#' (hash) symbol.

Structure of a Preprocessor directive

A preprocessor directive begins with the # character – only white-space characters can come before a preprocessor directive on a line. The directives may have additional tokens after them on the line. For example:

```
#example_directive token1 token2
```

Common Preprocessor directives

The following are some preprocessor directives that beginner programmers encounter.

#include Preprocessor directive

The structure of the #include directive is:

#include filename

The filename is the name of a standard library (if it is enclosed in angle brackets <>) or a custom C++ file (if it is enclosed in double quotes "").

When the compiler encounters the #include directive, it replaces the line with the contents of the file.

#define Preprocessor directive

The #define directive is used to define Macros and Symbolic Constants. The structure of the #define directive is:

#define identifier replacement-text

In the preprocessing phase, the compiler replaces any occurrences of identifier that don't occur as part of a string literal or a comment with the replacement-text.

#error Preprocessor directive

The structure of the #error directive is:

```
#error tokens
```

Here, the tokens are characters separated by white-spaces.

In the preprocessing phase, if the compiler encounters the #error preprocessor directive, it terminates the compilation and prints the tokens as error on the standard output.

Conditional inclusion Preprocessor directives

Conditional inclusion Preprocessor directives consist of the following:

```
#if
```

```
#ifdef
```

```
#ifndef
```

```
#elif
```

```
#else
```

```
#endif
```

These directives instruct the compiler to conditionally include parts of the source code for compilation. The following code provides an example:



Example

```
#include <iostream>

using namespace std;

#define PI 3.142

int main() {
    #ifdef PI
        cout << "The value of PI is: " << PI << endl;
    #else
        cout << "PI is not defined!" << endl;
    #endif
    return 0;
}
```

12.4 Macros in C++

A macro is a fragment of code which has been given a name. Whenever the name is used, it is replaced by the contents of the macro. There are two kinds of macros. They differ mostly in what they look like when they are used. Object-like macros resemble data objects when used, function-like macros resemble function calls.



Did you know?

- Macros are a piece of code in a program which is given some name. Whenever this name is encountered by the compiler the compiler replaces the name with the actual piece of code.
- The '#define' directive is used to define a macro.

Importance of macros

- Unless making a basic file you have to write **#include**, which is a macro that pastes the text contained in a file.
- With a name that gives the statement a meaning, its easier to understand the macro than the lengthy code behind it.



Example

```
#include<iostream>

using namespace std;

#define END 10

int main(){
    int i;
    for(i=0;i<=END;i++){
```

```

    cout<<i<<endl;
}
return 0;
}

```

Output

```

0
1
2
3
4
5
6
7
8
9
10

Process returned 0 (0x0)   execution time : 0.119 s
Press any key to continue.

```

Difference between Macro and Function

Macro	Function
It is not compiled, it is pre-processed.	It is not pre-processed but it is compiled.
Macros do not check for compilation error which leads to unexpected output.	Function checks for compilation error and there is a less chance of unexpected output.
Code length is increased.	Code length remains same.
Macros are faster in execution than function.	Functions are bit slower in execution.
Before compilation process the macro name is replaced by the macro value.	In a function call, transfer of control takes place.
Macros are useful when a small piece of code used multiple times in a program.	Functions are helpful when a large piece of code repeated number of times.

Macros vs. Templates

- Macro are used in C and C++ for replacement of numbers, small inline functions etc.
- Template is only available in C++ language. It is used to write small macro like functions etc.

Difference between Macros and Templates

Macro	Template
There is no type checking	There is full type checking
They are preprocessed	They are compiled
They can cause an unexpected result	No such unexpected results are obtained.



Example

//Program Macro as Function

```
#include <iostream>
```

```
using namespace std;
```

```
#define MIN(a,b) (((a)<(b)) ? a : b)
```

```
int main () {
```

```
    int i, j;
```

```
    cout<<"Enter Two numbers";
```

```
    cin>>i>>j;
```

```
    cout <<"The minimum is " << MIN(i, j) << endl;
```

```
    return 0;
```

```
}
```

Output

```
Enter Two numbers50
12
The minimum is 12

Process returned 0 (0x0)   execution time : 3.868 s
Press any key to continue.
```

Summary

- C++ supports a mechanism known as template to implement the concept of generic programming.
- Templates classes may be defined as the layout and operations for an unbounded set of related classes.
- Templates allows us to generate a family of classes or a family of functions to handle different data types.
- Macros are a piece of code in a program which is given some name. Whenever this name is encountered by the compiler the compiler replaces the name with the actual piece of code. The '#define' directive is used to define a macro.
- Template classes and functions eliminate code duplication for different types and thus make the program development easier and more manageable.

Keywords

Template: A template function may be defined as an unbounded functions. All the possible parameters to the function are not known in advance and a copy of the function has to be created as and when necessary.

Template Classes: Template classes may be defined as the layout and operations for an unbounded set of related classes.

Function Template: We write a generic function that can be used for different data types. Function templates are special functions that can operate with generic types.

Generic Programming in C++: Generic programming is a technique where generic types are used as parameters in algorithms so that they can work for a variety of data types.

Preprocessor in C++: As the name suggests Preprocessors are programs that process our source code before compilation. There are a number of steps involved between writing a program and executing a program in C / C++.

Self Assessment

1. Template is an example of compile time polymorphism.
 - A. True
 - B. False

2. How to declare a template?
 - A. temp
 - B. template < >
 - C. tem
 - D. all of above

3. The same template is used to generate many different instances, this can be done by
 - A. Functions
 - B. Template parameters
 - C. Operators
 - D. None of them

4. Recursion is a method in which the solution of a problem depends on _____
 - A. Larger instances of different problems
 - B. Larger instances of the same problem
 - C. Smaller instances of the same problem
 - D. Smaller instances of different problems

5. A template class can have _____
 - A. More than one generic data type
 - B. Only one generic data type
 - C. At most two data types
 - D. Only generic type of integers and not characters

6. What is the syntax to use explicit class specialization?
 - A. template <int> class myClass<>{ }
 - B. template <int> class myClass<int>{ }
 - C. template <> class myClass<>{ }
 - D. template <> class myClass<int>{ }

7. Which is the most significant feature that arises by using template classes?

- A. Code readability
- B. Ease in coding
- C. Code reusability
- D. Modularity in code

8. It is possible to inherit from a template class.

- A. True
- B. False

9. What will the output of following code

```
#include<iostream>
```

```
using namespace std;
```

```
template <typename T>
```

```
class base {
```

```
protected:
```

```
    int x=10,y=10;
```

```
};
```

```
template <typename T>
```

```
class derived : public base<T> {
```

```
public:
```

```
    int sum() { return this->x+this->y; }
```

```
};
```

```
int main() {
```

```
    derived<int> d;
```

```
    cout<<"Sum of x and y is"<<d.sum();
```

```
    return 0;
```

```
}
```

- A. 10 10
- B. 110
- C. 20
- D. 10

10. Which keyword is used to define the macros in c++?

- A. macro
- B. define
- C. #define
- D. none of the mentioned

11. Which symbol is used to declare the preprocessor directives?

- A. \$
- B. %
- C. *
- D. #

12. What is the output of this program?

```
#include <iostream>

using namespace std;

#define MIN(a,b) (((a)<(b)) ? a : b)

int main ()
{
    float i, j;
    i = 10.1;
    j = 10.01;
    cout << "The minimum is " << MIN(i, j) << endl;
    return 0;
}
```

- A. 10.01
- B. 10.1
- C. compile time error
- D. none of the mentioned

13. What is the output of this program?

```
#include <iostream>

using namespace std;

#define MIN(a,b) (((a)<(b)) ? a : b)

int main () {
    int i, j;
    i=90;
    j=10;
    cout <<"The minimum is " << MIN(i, j) << endl;
    return 0;
}
```

- A. The minimum is 0
- B. The minimum is 10
- C. The minimum is 90
- D. Compile time error

14. Which one is right option for function?

- A. It is not pre-processed but it is compiled.

- B. Code length remains same.
 - C. Functions are bit slower in execution.
 - D. All of above
15. Which one is right option for function?
- A. Macros are faster in execution than function.
 - B. Macros are useful when a small piece of code used multiple times in a program.
 - C. All of above
 - D. None of above

Review Questions

1. What is macro? How is it implemented in C++?
2. A template can be considered as a kind of macro. Then, what is the difference between them? Distinguish between overloaded functions and function templates.
3. How to implement recursion using template in C++.
4. A macro can be used as a function. Comment.
5. What are Templates?
6. Define preprocessor in C++.
7. Differentiate between macro and template.
8. Explain recursion with template function with suitable example
9. Write a program in C++ that demonstrate working of preprocessor in C++.
10. Write a program in C++ that demonstrate working of macro.

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. A | 2. B | 3. B | 4. C | 5. A |
| 6. D | 7. C | 8. A | 9. C | 10. C |
| 11. D | 12. A | 13. B | 14. D | 15. C |



Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.
Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.
Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.



Web Links

<https://www.codecademy.com/learn/learn-c-plus-plus>
<https://isocpp.org/wiki/faq/templates>
<https://www.startertutorials.com/blog/generic-programming-cpp.html>